

Software Requirements Specification

for

AI Based Personal Teacher Assistant

Version 1.0 approved

Prepared by **Group 12**

Supervised by Mr. Chathura Mahasen Bandara

Faculty of Technology

University of Colombo

<https://chatgpt.com/share/6999b84e-239c-800a-a699-f229e18104df>

<https://chatgpt.com/c/699ae407-8a80-8324-aa69-270c89a62c09>

Contents

Contents

1. Introduction	5
1.1 Purpose	5
1.2 Document Conventions	6
1.3 Intended Audience and Reading Suggestions.....	7
1.4 Product Scope	7
1.5 Reference	8
2. Overall Description.....	9
2.1 Product Perspective	9
2.2 Product Functions	10
2.2.1 High-Level System Diagram	11
2.3 User Classes and Characteristics.....	12
2.3.1 Use Case Diagram	12
2.4 Operating Environment.....	14
2.5 Design and Implementation Constraints	15
2.5.1 Technical Constraints	15
2.5.2 Security Constraints	16
2.5.3 Regulatory and Institutional Constraints	16
2.5.4 Programming and Design Standards.....	16
2.6 User Documentation.....	16
2.6.1 User Manual.....	17
2.6.2 Quick Start Guide	17
2.6.3 Online Help System.....	17
2.6.4 Technical Documentation	17
2.6.5 Training Materials	17
2.7 Assumptions and Dependencies.....	18
2.7.1 Assumptions.....	18
2.7.2 Dependencies	19
2.8 Data Flow Diagram - Level 1	20
3. External Interface Requirements	21

3.1 User Interfaces	21
3.2 Hardware Interfaces.....	21
3.3 Software Interfaces	22
3.4 Communication Interfaces.....	24
4. System Features	26
4.1 User Authentication and Access Control	26
4.1.1 Description and Priority	26
4.1.2 Stimulus / Response Sequences	26
4.1.3 Functional Requirements.....	26
4.2 Answer Script Upload and Management.....	27
4.2.1 Description and Priority	27
4.2.2 Stimulus / Response Sequences	27
4.2.3 Functional Requirements.....	27
4.3 AI-Based Answer Evaluation	27
4.3.1 Description and Priority	27
4.3.2 Stimulus / Response Sequences	28
4.3.3 Functional Requirements.....	28
4.4 Low-Confidence Flagging and Manual Review	28
4.4.1 Description and Priority	28
4.4.2 Stimulus / Response Sequences	28
4.4.3 Functional Requirements.....	29
4.5 Report Generation and Analytics Dashboard	29
4.5.1 Description and Priority	29
4.5.2 Stimulus / Response Sequences	29
4.5.3 Functional Requirements.....	29
5. Other Nonfunctional Requirements	30
5.1 Performance Requirements	30
5.2 Safety Requirements.....	30
5.3 Security Requirements.....	30
5.4 Software Quality Attributes	31
5.5 Business Rules.....	31
6. Other Requirements	32

Table of Tables

Table 1.....	4
Table 2.....	5
Table 3.....	6
Table 4.....	26
Table 5.....	26
Table 6.....	26
Table 7.....	26
Table 8.....	27
Table 9.....	27
Table 10.....	27
Table 11.....	28
Table 12.....	28
Table 13.....	28
Table 14.....	28
Table 15.....	29
Table 16.....	29
Table 17.....	29
Table 18.....	29

Table of Figures

Group Members

Table 1

	STUDENT NAME	STUDENT NO.	CONTACT NO.	EMAIL ADDRESS
1	K. G. Amila Udayanga	2021t01214	0769129718	2021t01214@stu.cmb.ac.lk
2	P. S. M. Disanayaka	2021t01201	0758670845	2021t01201@stu.cmb.ac.lk
3	R. L. S. J. Abeygunawardhana	2021t01190	0705576580	2021t01190@stu.cmb.ac.lk

Revision History

Table 2

Name	Date	Reason For Changes	Version

--	--	--	--

1.Introduction

1.1 Purpose

The purpose of this project is to develop **MarkNex**, an AI-Based Personal Teacher Assistant designed to automate the marking of handwritten answer scripts and generate personalized feedback for students and teachers.

In Sri Lankan schools, manual marking of answer scripts is time-consuming, repetitive, and prone to inconsistencies. Teachers often struggle to provide timely and detailed feedback due to heavy workloads. Existing digital grading systems mainly rely on traditional Optical Character Recognition (OCR) methods, which perform poorly when interpreting handwritten responses and do not provide adaptive, concept-level feedback.

This project aims to address these challenges by developing a multimodal Large Language Model (LLM)-powered system capable of:

- Reading handwritten answers directly from scanned images.
- Converting handwritten responses into structured digital text.
- Evaluating answers based on predefined marking schemes.
- Assigning marks with a confidence score.
- Flagging low-confidence answers for manual teacher review.
- Generating personalized student reports.
- Providing teachers with class-level analytics and insights.

The system is intended to:

- Reduce teacher workload.
- Improve grading efficiency and consistency.
- Provide actionable learning insights.
- Enhance overall teaching and learning outcomes.

1.2 Document Conventions

This document follows the IEEE 830-1998 Software Requirements Specification (SRS) standard. The following conventions are used throughout this document.

1. Font and Formatting

- Standard font (e.g. Calibri).
- Font size (e.g., 11pt for body text, 14pt for headings).
- Bold for section titles, italics for emphasis.

2. Numbering and Sectioning:

- Follows IEEE 830-1998 SRS standard structure (e.g., 1. Introduction, 2. Overall Description, etc.).
- Subsections numbered (e.g., 1.1, 1.2).

3. Terminology and Definitions:

- Specific terms and abbreviations are defined in a glossary.
- Acronyms like UI (User Interface), API (Application Programming Interface) are explained.

4. Diagrams and Figures:

- Labeled with figure numbers (e.g., "Figure 1: Use Case Diagram").
- Included with a description below the figure.

5. Consistency:

- Uniform language and formatting across the document.
- Use of active voice and clear sentence structures.

All requirements are categorized by priority levels, where High Priority requirements must be implemented in the initial version, and Low Priority requirements can be considered for future updates. Figures, tables, and diagrams follow a sequential numbering format (e.g., Figure 1, Table 1).

1.3 Intended Audience and Reading Suggestions

This section outlines the various user groups who will interact with the **MarkNex**, an AI-Based Personal Teacher Assistant system and the relevant sections of this document that they should focus on.

Table 3

User Group	Purpose of Reading the SRS	Recommended Sections to Read
Project Supervisors & Academic Evaluators	To review and validate the functional and non-functional requirements of the system.	Purpose, Scope, System Features, Assumptions and Constraints
Project Development Team (AI/ML Developers, Backend	To use as a reference for system design, implementation, and testing.	Section 3: Specific Requirements, Functional

Developers, Frontend Developers)		Requirements, Non-Functional Requirements
School Teachers (End Users)	To understand the capabilities and limitations of the system.	Section 1: Introduction, Section 2: Overall Description
System Testers & QA Team	To derive test cases from functional and non-functional requirements.	Functional Requirements, Performance and Reliability Requirements
Future Researchers & Developers	To support system enhancement, scalability, or integration with other educational platforms.	Overall System Architecture, System Features, Assumptions and Constraints

Recommended Sections to Read me part ek add krnn doc hadala iwar unata passe

1.4 Product Scope

MarkNex is an AI-powered web-based application designed to assist teachers in automatically evaluating handwritten Handwritten answer scripts.

Core Scope of the Product

The system will:

1. Allow teachers to upload scanned images of student answer sheets.
2. Use a multimodal Large Language Model (LLM) to interpret handwritten responses.
3. Convert image-based answers into structured text.
4. Compare responses against predefined marking schemes and model answers.
5. Automatically assign marks and generate confidence scores.
6. Flag low-confidence answers for manual review.
7. Generate:
 - Individual student feedback reports.
 - Class-level performance analytics.
8. Provide dashboards for visualization of:
 - Topic-wise performance.
 - Frequently misunderstood questions.
 - Student strengths and weaknesses.

System Boundaries

The system will:

- Support Handwritten question papers (initial version).

- Operate as a web-based platform.
- Include teacher authentication and secure access.
- Store evaluation results and reports in a database.

The system will NOT:

- Replace teachers entirely in the grading process.
- Support all subjects in the initial release.
- Guarantee 100% automated grading without human oversight.

1.5 Reference

IEEE 830-1998 Software Requirements Specification Standard.

2. Overall Description

2.1 Product Perspective

MarkNex is a standalone web-based AI-powered system developed to automate the evaluation of handwritten Grade 6 Mathematics answer scripts.

The system integrates:

- A **multimodal Large Language Model (LLM)** for handwritten answer interpretation.
- A backend server for data processing and report generation.
- A database for storing student records, evaluation results, and reports.
- A frontend dashboard for teachers.

MarkNex operates as an independent academic support system but can be extended in the future to integrate with:

- School Learning Management Systems (LMS)
- Student Information Systems
- Cloud storage platforms

The system follows a **human-in-the-loop approach**, where AI performs automated evaluation and teachers validate low-confidence responses.

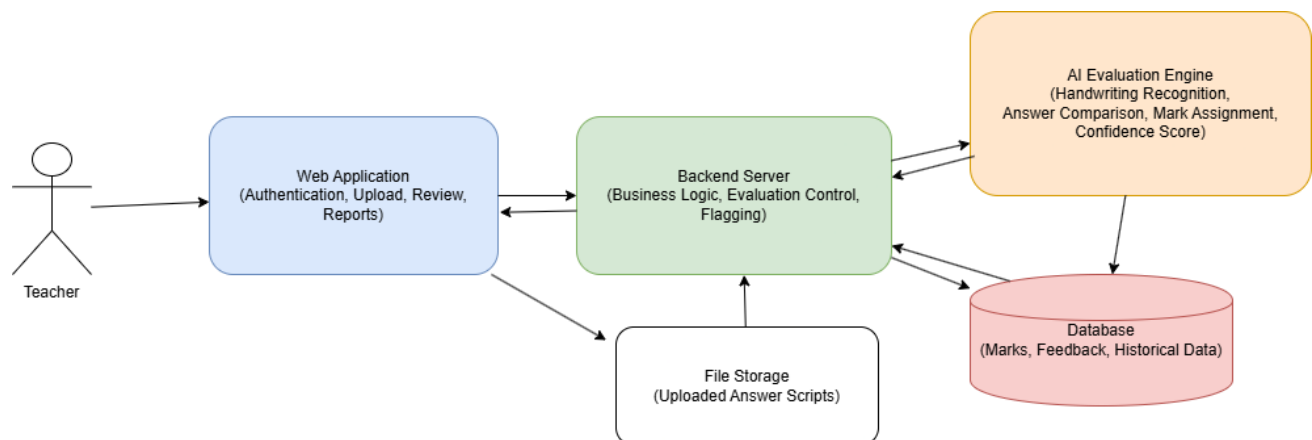
2.2 Product Functions

The main functions of the MarkNex system include:

1. **User Authentication**
 - Secure login for teachers.
 - Session management.
2. **Answer Script Upload**
 - Upload scanned images (PDF/JPG/PNG).
 - Associate scripts with student records.
3. **AI-Based Answer Evaluation**
 - Interpret handwritten responses.
 - Convert image data into structured digital text.
 - Compare answers with marking schemes.
 - Assign marks automatically.

- Generate confidence score.
- 4. **Low-Confidence Flagging**
 - Identify uncertain evaluations.
 - Notify teacher for manual review.
- 5. **Manual Review Panel**
 - Display flagged answers.
 - Allow teacher to adjust marks.
- 6. **Report Generation**
 - Student-level feedback reports.
 - Topic-wise performance summary.
 - Class-level analytics dashboard.
- 7. **Data Storage**
 - Store marks, feedback, and reports.
 - Maintain historical performance data

2.2.1 High-Level System Diagram



The **MarkNex system** follows a structured and sequential workflow to ensure secure, accurate, and efficient automated evaluation of handwritten answer scripts.

The process begins with user authentication. The teacher accesses the web-based interface and enters valid login credentials. The backend server verifies these credentials against the stored database records. After successful verification, the system establishes a secure session.

Next, the teacher uploads scanned answer scripts in supported formats such as PDF, JPG, or PNG. The system associates each script with relevant metadata, including student identification details, subject, examination information, and marking scheme references. It stores the uploaded files in a secure file storage system and records the corresponding metadata in the central database to ensure traceability and structured processing.

After the upload, the backend server sends the script to the AI Evaluation Engine for automated assessment. The AI component preprocesses the image and performs handwriting recognition to convert the scanned content into structured digital text. It then compares the extracted text with predefined marking schemes using rule-based logic and semantic similarity techniques. Based on this comparison, the system assigns marks to each question, calculates the total score, and generates a confidence score to measure the reliability of the evaluation.

If the confidence score exceeds the predefined threshold, the system automatically finalizes the marks and stores them in the database. If the confidence score falls below the acceptable level, the system flags the evaluation for manual review. It notifies the teacher and provides access to a review interface. The interface displays the original script, extracted text, AI-generated marks, and marking scheme references. The teacher can confirm, modify, or override the assigned marks before submitting the final evaluation.

After completing the evaluation, the system generates detailed reports. These reports include student-level feedback, topic-wise performance summaries, and class-level analytical dashboards. The system securely stores all finalized marks, feedback, confidence scores, and historical performance data in the database to support long-term academic tracking and performance analysis.

This workflow integrates artificial intelligence with human oversight. It improves efficiency while maintaining fairness and reliability in the assessment process.

2.3 User Classes and Characteristics

1. Teacher (Primary User)

- Moderate computer literacy.
- Familiar with exam marking process.
- Responsible for uploading scripts and reviewing flagged answers.
- Needs simple and intuitive interface.

2. System Administrator (Optional/Future Role)

- Technical knowledge.
- Manages system settings, user accounts, and monitoring.

3. Researchers/Developers

- Technical expertise in AI/ML and software engineering.

- Monitor system performance and model accuracy.

2.3.1 Use Case Diagram



The **Use Case Diagram** represents the functional requirements of the MarkNex system and illustrates how users and external systems interact with the application. It highlights the primary actors, the system boundary, and the key use cases.

Actors

1. **Teacher** – The primary actor who interacts with the system to perform all teaching-related functions such as uploading answer scripts, reviewing AI-generated evaluations, adjusting marks, and generating reports.
2. **AI Evaluation Engine** – An external system responsible for evaluating the uploaded scripts, generating marks, and calculating confidence scores.

System Boundary

The diagram defines the **MarkNex System** as the boundary within which all internal functionalities operate. This helps to distinguish between external actors and the internal system processes.

Use Cases and Flow

1. **Login** – Allows the teacher to securely access the system by providing credentials.
2. **Upload Answer Script** – Enables the teacher to upload scanned answer sheets, linking them with student and exam details.
3. **Associate Student Details** – Connects each uploaded script with the relevant student record and metadata.
4. **Evaluate Answer Script** – Performed by the AI Evaluation Engine, converting handwritten scripts to digital text and comparing them to the marking scheme.
5. **Generate Confidence Score** – AI calculates a confidence level to indicate the reliability of its evaluation.
6. **Flag Low-Confidence Answers** – The system identifies uncertain evaluations that require teacher verification.
7. **Review Flagged Answers** – Teachers manually check flagged answers, compare AI-assigned marks, and confirm or adjust them.
8. **Adjust Marks** – Teachers can modify marks during manual review before final submission.
9. **Generate Reports** – Produces detailed student feedback and performance summaries.

10. **View Analytics Dashboard** – Displays class-level analytics, trends, and performance insights for the teacher.

Relationships

- The **Teacher** is associated with all use cases that involve human interaction: login, upload, review, adjustment, and dashboard access.
- The **AI Evaluation Engine** is associated with **Evaluate Answer Script** and **Generate Confidence Score**, report generation, representing the automated processing performed outside the teacher's direct control.

2.4 Operating Environment

The MarkNex system will operate in the following environment:

Hardware Requirements

- Server: Cloud-based or local server.
- Client: Desktop/Laptop with internet connection.

Software Requirements

- Backend: Node.js / Python (Django/Flask)
- Frontend: React / HTML, CSS, JavaScript
- Database: MySQL / PostgreSQL
- AI Model Integration: API-based multimodal LLM
- Web Browser: Chrome, Edge, Firefox (latest versions)

Operating Systems

- Server: Linux/Windows
- Client: Windows, macOS

The following constraints limit the design and implementation options for the MarkNex system:

2.5 Design and Implementation Constraints

The following constraints limit the design and implementation options for the MarkNex system:

2.5.1 Technical Constraints

1. AI Model Dependency

- The system depends on a multimodal Large Language Model (LLM) for handwritten answer interpretation.
- Model performance directly affects grading accuracy.
- API availability and response time may impact system performance.

2. Technology Stack Constraints

- Backend must be implemented using approved technologies (e.g., Python with Django/Flask or Node.js).
- Frontend must be developed using modern web technologies (HTML5, CSS3, JavaScript frameworks).
- Database must use relational DBMS (MySQL or PostgreSQL).

3. Hardware Limitations

- Cloud server resource limitations (CPU, RAM).
- Image processing and AI inference require high computational resources.
- File upload size limitations for scanned answer scripts.

4. Integration Constraints

- AI model integration must follow secure API communication protocols (HTTPS).
- System must support standard file formats: PDF, JPG, PNG.

2.5.2 Security Constraints

1. User authentication and authorization must be implemented.
2. Secure communication using HTTPS protocol.
3. Protection of student data according to academic data privacy policies.
4. Role-based access control for teachers and administrators.
5. Secure database storage with encryption where applicable.

2.5.3 Regulatory and Institutional Constraints

1. Must comply with school-level data protection policies.

2. Must align with the Sri Lankan educational curriculum.
3. Ethical use of AI in student assessment must be ensured.
4. The system must include human oversight (teacher review for low-confidence results).

2.5.4 Programming and Design Standards

1. Code must follow modular architecture.
2. RESTful API standards must be followed.
3. Clear documentation and version control must be maintained.
4. System must follow standard UI/UX usability guidelines.
5. Development must follow structured software engineering practices (e.g., Agile methodology).

2.6 User Documentation

The MarkNex system will be delivered with comprehensive user documentation to ensure that all users can effectively understand, operate, and maintain the system. The documentation is designed to support teachers, system administrators, and technical personnel by providing clear operational guidelines, technical references, and training resources.

2.6.1 User Manual

A detailed user manual will be provided as the primary reference document for teachers and authorized users. This manual will describe the overall system overview, including its purpose and key functionalities. It will explain the login and authentication process, step-by-step instructions for uploading scanned answer scripts, procedures for reviewing AI-flagged answers, and guidelines for generating and downloading evaluation reports. In addition, the manual will include a dashboard interpretation guide to help users understand performance analytics, summaries, and student insights. The user manual will be available as a PDF document and as a digital soft copy accessible within the system.

2.6.2 Quick Start Guide

A Quick Start Guide will be developed to assist first-time users in quickly understanding how to operate the system. This guide will provide simple step-by-step instructions for initial login and basic system usage. It will outline the overall workflow of uploading scripts, reviewing results, and generating reports. It will also include solutions to common troubleshooting issues that users may encounter. The Quick Start Guide will be available as a printable PDF and as a web-based help page integrated into the system.

2.6.3 Online Help System

An Online Help System will be embedded within the MarkNex interface to provide real-time assistance. This will include tooltips displayed when users hover over specific features, context-sensitive help messages that appear based on user actions, and an FAQ section accessible from the system dashboard. This integrated support system ensures that users can receive immediate guidance without leaving the application.

2.6.4 Technical Documentation

Technical documentation will be prepared for system administrators, developers, and IT support staff responsible for maintaining and deploying the system. This documentation will include a detailed system architecture overview explaining the structural design of the application. It will provide a database schema description outlining data structures and relationships. Additionally, API documentation will describe endpoints, request formats, and response structures using a standard documentation format such as Swagger/OpenAPI. A deployment guide will also be included to explain installation, configuration, and hosting procedures. The technical documentation will be available in PDF or Markdown format.

2.6.5 Training Materials

Training materials will be provided to ensure effective system adoption among teachers. These materials will include structured presentation slides for training sessions, a demonstration video explaining the complete system workflow, and a sample answer script walkthrough to illustrate the marking and evaluation process. These resources will help users gain practical knowledge and confidence in using the MarkNex system efficiently.

2.7 Assumptions and Dependencies

2.7.1 Assumptions

This section outlines the key assumptions and dependencies identified during the preparation of the Software Requirements Specification (SRS) for the MarkNex system. These assumptions define the expected conditions under which the system will operate. If any of these assumptions are invalidated, the system requirements, design, or implementation strategy may require revision.

Technical Assumptions

It is assumed that the multimodal Large Language Model (LLM) selected for handwritten answer interpretation will provide sufficiently accurate and reliable output when evaluating handwritten answer scripts. The system assumes that the AI model is capable of interpreting handwritten content without relying exclusively on traditional Optical Character Recognition (OCR) methods.

It is further assumed that the external API services required for AI model integration will remain stable, accessible, and compatible throughout the development and deployment phases. The system is expected to operate in a stable internet-connected environment to support AI processing and data synchronization.

Additionally, it is assumed that uploaded answer scripts will be clearly scanned, properly aligned, and legible. The system assumes that image quality will meet minimum acceptable standards. Poor image quality, distortions, shadows, or blurred handwriting may negatively affect evaluation accuracy.

Operational Assumptions

It is assumed that teachers using the MarkNex system possess basic computer literacy, including the ability to log in, upload files, navigate dashboards, and download reports.

The system follows a human-in-the-loop approach, assuming that teachers will review and validate answers flagged as low-confidence by the AI model. Final grading authority remains with the teacher.

It is also assumed that schools will permit the use of AI-assisted grading systems within their academic processes. Furthermore, marking schemes and model answers required for evaluation will be properly structured and available in digital format to support automated analysis.

Academic Assumptions

The initial implementation of the system is aligned with the Sri Lankan national curriculum and the approved project charter.

The evaluation logic assumes that answers follow structured solution formats, including clear step-by-step problem-solving approaches that are typical in structured assessments.

It is assumed that the system will primarily be used for both formative and summative assessments within a controlled school environment.

Development Assumptions

It is assumed that all development team members will complete their assigned responsibilities as defined in the approved project charter and timeline.

The availability of appropriate datasets for training, validation, and testing of the AI model is assumed. These datasets are expected to represent realistic student handwriting samples and curriculum-aligned mathematical problems.

It is also assumed that the development timeline is aligned with the university's final year project schedule and that required academic milestones will be met as planned.

2.7.2 Dependencies

The MarkNex system relies on several external components, technologies, and institutional approvals. Changes or failures in these dependencies may directly affect system performance, availability, or functionality.

Third-Party Dependencies

The system depends on a multimodal Large Language Model (LLM) service provider for handwritten answer interpretation and AI-based evaluation.

It also depends on a cloud hosting platform for application deployment and scalable computing resources. A web server and database management system are required for handling user requests and data storage.

Secure communication between system components relies on API frameworks, particularly RESTful APIs.

If any third-party service modifies pricing models, API structures, access policies, or availability, system functionality and sustainability may be impacted.

Software Dependencies

The backend implementation depends on a selected framework such as Python (Django) or Node.js. The frontend depends on technologies such as React or standard web technologies including HTML, CSS, and JavaScript.

A Relational Database Management System (RDBMS), such as MySQL or PostgreSQL, is required for structured data storage. The system also depends on PDF and image processing libraries for handling uploaded answer scripts.

Future updates, version incompatibilities, or deprecation of these technologies may affect system maintenance and long-term stability.

Infrastructure Dependencies

The system requires reliable internet connectivity to support AI inference and cloud-based processing. Adequate server computational resources are necessary to handle AI model execution efficiently.

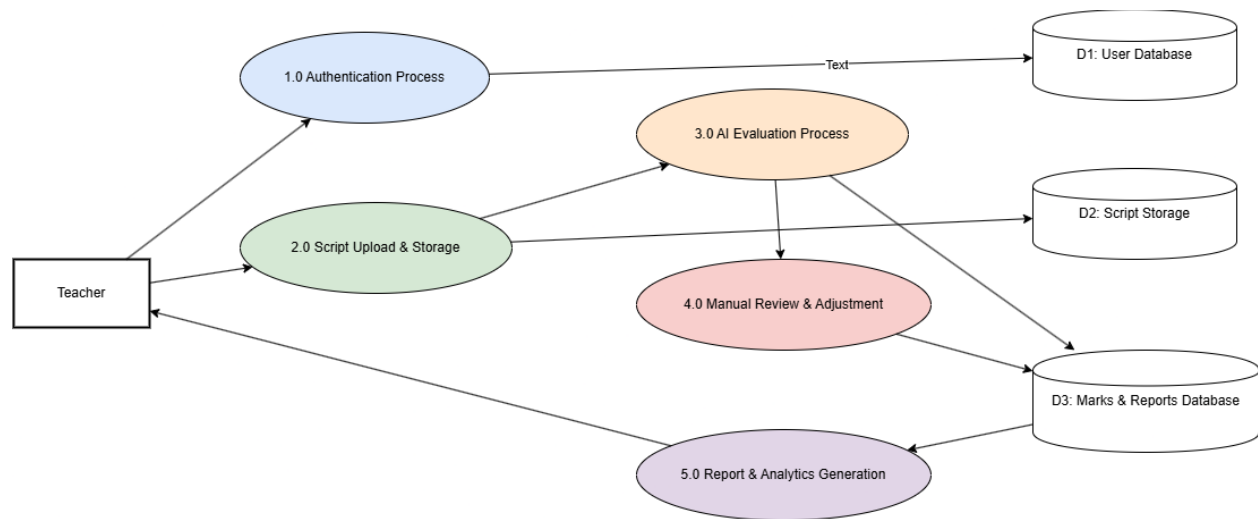
A secure hosting environment with HTTPS support is required to ensure data protection and secure communication.

Institutional Dependencies

The implementation of the MarkNex system depends on approval from the Department of ICT and the Faculty of Technology.

The availability of real or sample student answer scripts is necessary for system testing and validation. If student data is used, appropriate ethical approval and data protection compliance must be obtained before deployment.

2.8 Data Flow Diagram - Level 1



The Data Flow Diagram (DFD) Level 1 for the MarkNex system illustrates how data moves between external entities, internal processes, and data stores within the system. It decomposes the overall system into major functional processes and clearly shows how information flows during authentication, script evaluation, review, and report generation.

The process begins when the teacher provides login credentials to the Authentication Process. The system verifies these credentials by interacting with the User Database. If the credentials are valid, the system grants access and allows the teacher to proceed. This process ensures secure access control and protects sensitive academic data.

After authentication, the teacher uploads scanned answer scripts through the Script Upload and Storage process. The system stores the uploaded files in the Script Storage data store and records associated metadata, such as student information and exam details. Once the storage process is complete, the system forwards the script data to the AI Evaluation Process for automated marking.

The AI Evaluation Process analyzes the uploaded script, converts handwritten content into digital text, compares it with the marking scheme, and assigns marks. The system stores the generated marks and confidence scores in the Marks and Reports Database. If the confidence level falls below the defined threshold, the system directs the data to the Manual Review and Adjustment process, where the teacher reviews flagged answers and modifies marks if necessary. The system then updates the database with the finalized results.

Finally, the Report and Analytics Generation process retrieves evaluation data from the Marks and Reports Database and generates structured reports and performance analytics. The system delivers these reports to the teacher for review and academic decision-making. Overall, the DFD Level 1 clearly demonstrates how MarkNex manages data input, processing, storage, and output while maintaining controlled data movement between processes and data stores.

3. External Interface Requirements

3.1 User Interfaces

Screenshots of system UI

3.2 Hardware Interfaces

This section describes the interaction between the MarkNex software system and the hardware components required for its operation. The system is primarily web-based and does not require specialized hardware beyond standard computing devices and optional scanning equipment. However, proper hardware support is necessary to ensure efficient system performance and accurate AI-based evaluation.

Supported Hardware Devices

The MarkNex system is designed to operate on commonly available computing devices. It supports desktop computers and laptops used by teachers to access the web-based interface. These devices must have a modern web browser and stable internet connectivity to ensure smooth interaction with the system.

The system is deployed on cloud server infrastructure, which provides the computational resources required for data storage, processing, and AI model execution. The server environment may include high-performance CPU and, where necessary, GPU resources to handle AI inference tasks efficiently.

Optionally, flatbed scanners may be used to digitize handwritten answer scripts. Although scanning is not handled directly by the system, scanned documents must be converted into supported digital formats such as PDF, JPG, or PNG before being uploaded.

Hardware Interaction Characteristics

Teachers interact with scanner hardware externally to scan student answer scripts. The scanning process occurs outside the system environment. Once scanning is completed, the digitized answer scripts are uploaded to the MarkNex system through a web browser interface.

Upon upload, the server processes the images using its computational resources. The AI model performs handwritten answer interpretation and evaluation using CPU and/or GPU capabilities available within the cloud infrastructure.

The system does not directly control or manage hardware devices; instead, it relies on standard input/output mechanisms through web technologies to ensure compatibility and flexibility across different hardware configurations.

3.3 Software Interfaces

This section defines the interfaces between the MarkNex system and external or supporting software components. These interfaces ensure seamless communication between the application, operating systems, database systems, AI services, and web browsers.

Operating System Interface

The MarkNex system is designed to operate independently of specific operating system constraints.

On the server side, the system shall be compatible with commonly used server operating systems such as Linux and Windows Server. The backend services, database management system, and AI integration components shall function reliably within these environments.

On the client side, the system shall support widely used operating systems including Windows and macOS. Since MarkNex is a web-based application, users interact with the system through a web browser. Therefore, no OS-specific dependencies shall affect user interaction, and no additional software installation is required on the client device beyond a supported browser.

Database Interface

The MarkNex system shall interface with a Relational Database Management System (RDBMS), such as MySQL or PostgreSQL, for structured data storage and retrieval.

The database shall manage and store the following types of data:

- Student details
- Uploaded scripts metadata
- Marks and AI confidence scores
- Generated feedback reports
- User authentication data

Communication between the application and the database shall occur using structured SQL queries. Where applicable, an Object-Relational Mapping (ORM) framework may be used to manage database operations, ensuring secure, efficient, and maintainable data access.

AI/ML Model Interface

The MarkNex system integrates with an external Multimodal Large Language Model (LLM) to perform handwritten answer interpretation and automated evaluation.

The data transmitted to the AI module shall include:

- Encoded image data of uploaded answer scripts
- The corresponding marking scheme
- Model answers for reference

The data received from the AI module shall include:

- Extracted and interpreted text from handwritten content
- Assigned marks for each answer
- Confidence scores indicating evaluation reliability
- Explanatory feedback or reasoning for the assigned marks

Communication between the server and the AI model shall occur via RESTful API calls. Data exchange shall use JSON-formatted request and response messages over secure HTTPS connections to ensure data integrity and confidentiality.

Web Browser Interface

The MarkNex system shall support modern web browsers to ensure accessibility and compatibility.

Supported browsers include:

- Google Chrome (latest version)
- Microsoft Edge (latest version)
- Mozilla Firefox (latest version)

The system shall not require additional browser plugins, extensions, or third-party installations. All functionality shall be accessible using standard web technologies such as HTML, CSS, and JavaScript.

Shared Data Across Components

Certain key data elements shall be shared across multiple system components to ensure consistent processing and reporting. These include:

- Student ID
- Question ID
- Marks
- Confidence Score
- Topic-wise performance metrics

Data sharing between components shall occur through secure database persistence and API-based data transfer mechanisms. The system shall not use unsecured global memory or unsafe data-sharing practices. All data exchanges shall follow secure and structured communication standards to maintain system integrity and data protection.

3.4 Communication Interfaces

This section describes the communication requirements of the MarkNex system, including network protocols, security mechanisms, data transfer standards, synchronization processes, and optional notification services. These requirements ensure secure, reliable, and efficient data exchange between system components.

Network Communication

The MarkNex system requires stable internet connectivity for proper operation, particularly for AI processing and cloud-based deployment.

The system shall use standard web communication protocols, including HTTP and HTTPS. All communication between client devices, servers, and external AI services shall follow RESTful API principles for structured and scalable integration.

Data exchanged between components shall be formatted using JSON (JavaScript Object Notation) to ensure interoperability and readability.

All communications shall use HTTPS encryption to protect sensitive academic data during transmission.

Security Requirements

To ensure data protection and secure system operation, the following security measures shall be implemented:

- TLS (Transport Layer Security) encryption for all client-server communication.
- Token-based authentication mechanisms such as JSON Web Tokens (JWT) for secure user session validation.
- Role-based access control (RBAC) to restrict system functionalities based on user roles (e.g., teacher, administrator).
- Secure session management to prevent session hijacking and unauthorized access.
- Encrypted password storage using hashing and salting techniques to prevent credential compromise.

These measures ensure confidentiality, integrity, and secure access control within the system.

Data Transfer Requirements

The system shall enforce a configurable maximum file upload size limit to prevent server overload and ensure performance stability.

The system shall support batch uploading of multiple answer scripts to improve operational efficiency for teachers handling large class sizes.

Where necessary, efficient image compression techniques may be applied before AI processing to optimize bandwidth usage and reduce processing time without significantly affecting image quality.

Synchronization Mechanisms

AI-based evaluation shall operate asynchronously. After uploading scripts, users shall not be required to wait on a locked interface. Instead, the system shall process submissions in the background.

The system shall display clear processing status indicators (e.g., “Processing,” “Completed,” “Review Required”) to inform users about evaluation progress.

Database transactions shall follow atomicity and consistency principles to ensure accurate storage of marks and evaluation data.

An automatic save mechanism shall be implemented when teachers review and modify marks, ensuring that no reviewed data is lost due to unexpected interruptions.

Email or Notification Services (Optional / Future Enhancement)

As a future enhancement, the system may include email notifications to inform teachers when evaluations are completed.

Additionally, the system shall support downloadable PDF reports for record-keeping and academic documentation purposes.

These communication features will enhance usability, transparency, and workflow efficiency in future versions of the MarkNex system.

4. System Features

This section describes the core functional features of the MarkNex system. Functional requirements are organized according to major system features. Each feature includes a description, priority level, stimulus/response sequences, and uniquely identified functional requirements.

4.1 User Authentication and Access Control

4.1.1 Description and Priority

This feature enables secure user authentication, session management, and role-based access control for teachers and administrators. It ensures that only authorized users can access the system and perform permitted actions.

Table 4

Item	Details
Description	Enables secure user authentication, session management, and role-based access control for Teachers and Admins.
Priority	High
Benefit	9
Penalty if Absent	9
Cost	4
Risk	3

4.1.2 Stimulus / Response Sequences

Table 5

Stimulus	System Response
Teacher enters username and password	System validates credentials and grants access to dashboard
Invalid login attempt	System displays error message and denies access
Session timeout	System logs out user and redirects to login page

4.1.3 Functional Requirements

Table 6

Requirement ID	Functional Requirement
REQ-AUTH-1	Provide login interface for registered users
REQ-AUTH-2	Validate user credentials against database
REQ-AUTH-3	Deny access for invalid credentials and display error message
REQ-AUTH-4	Implement secure password storage using hashing and salting
REQ-AUTH-5	Automatically log out users after inactivity period
REQ-AUTH-6	Support role-based access control (Teacher/Admin)

4.2 Answer Script Upload and Management

4.2.1 Description and Priority

Table 7

Item	Details
Description	Enables secure user authentication, session management, and role-based access control for Teachers and Admins.
Priority	High
Benefit	9

Penalty if Absent	9
Cost	4
Risk	3

4.2.2 Stimulus / Response Sequences

Table 8

Stimulus	System Response
Upload PDF/JPG/PNG file	Validate format and securely store file
Upload unsupported format	Display file format error message
Upload multiple scripts	Process scripts sequentially or batch mode

4.2.3 Functional Requirements

Table 9

Requirement ID	Functional Requirement
REQ-UP-1	Allow PDF, JPG, PNG file uploads
REQ-UP-2	Reject unsupported formats
REQ-UP-3	Enforce configurable maximum file size
REQ-UP-4	Associate scripts with student identifiers
REQ-UP-5	Securely store uploaded scripts
REQ-UP-6	Display upload success confirmation

4.3 AI-Based Answer Evaluation

4.3.1 Description and Priority

Table 10

Item	Details
Description	Uses multimodal LLM to extract handwritten text, evaluate answers, assign marks, and generate confidence scores.
Priority	High
Benefit	9
Penalty	9
Cost	8
Risk	7

4.3.2 Stimulus / Response Sequences

Table 11

Stimulus	System Response
Script uploaded	Send script to AI module
AI returns result	Store marks and confidence scores
AI processing failure	Log error and notify teacher

4.3.3 Functional Requirements

Table 12

Requirement ID	Functional Requirement
REQ-AI-1	Send uploaded image data to AI module
REQ-AI-2	Convert handwritten answers into structured digital text
REQ-AI-3	Compare extracted text with marking scheme
REQ-AI-4	Assign marks per question
REQ-AI-5	Generate confidence score
REQ-AI-6	Store evaluation results in database
REQ-AI-7	Handle AI errors gracefully and notify user

4.4 Low-Confidence Flagging and Manual Review

4.4.1 Description and Priority

Table 13

Item	Details
Description	Flags low-confidence answers for teacher review to ensure reliability and fairness.
Priority	High
Benefit	8
Penalty	8
Cost	5
Risk	5

4.4.2 Stimulus / Response Sequences

Table 14

Stimulus	System Response
Confidence score below threshold	Flag answer for manual review
Teacher modifies marks	Update database and recalculate total marks

4.4.3 Functional Requirements

Table 15

Requirement ID	Functional Requirement
REQ-REV-1	Allow configuration of confidence threshold
REQ-REV-2	Automatically flag low-confidence answers
REQ-REV-3	Display flagged answers in review panel
REQ-REV-4	Allow teachers to edit assigned marks
REQ-REV-5	Record manual modifications with timestamps
REQ-REV-6	Recalculate total marks automatically

4.5 Report Generation and Analytics Dashboard

4.5.1 Description and Priority

Table 16

Item	Details
Description	Generates student reports and class-level analytics dashboard.
Priority	High
Benefit	9
Penalty	7
Cost	6
Risk	4

4.5.2 Stimulus / Response Sequences

Table 17

Stimulus	System Response
Evaluation completed	Generate individual student report
Teacher selects analytics view	Display topic-wise and class-level metrics

4.5.3 Functional Requirements

Table 18

Requirement ID	Functional Requirement
REQ-REP-1	Generate individual student reports
REQ-REP-2	Highlight incorrect answers and weak topics
REQ-REP-3	Generate class-level summary reports
REQ-REP-4	Display topic-wise performance metrics
REQ-REP-5	Allow report download in PDF format
REQ-REP-6	Allow viewing historical performance records

5. Other Nonfunctional Requirements

5.1 Performance Requirements

The MarkNex system shall meet the following performance requirements:

1. The system shall authenticate users within **3 seconds** under normal operating conditions.
2. The system shall acknowledge uploaded answer scripts within **5 seconds**.
3. The AI evaluation module shall process and generate results within **90 seconds per script**, depending on script size and complexity.
4. The system shall support at least **50 concurrent users** without significant degradation in response time.
5. The system shall support batch processing of multiple answer scripts.
6. The system shall retrieve student reports and analytics within **5 seconds**.
7. The system shall maintain a minimum **99% system availability**, excluding scheduled maintenance.
8. The system should support horizontal scaling to accommodate increasing workloads.
9. The database shall use indexing and optimization techniques to ensure efficient data retrieval.

5.2 Safety Requirements

The MarkNex system must ensure safe handling of academic data and prevent data loss:

1. The system shall automatically back up evaluation data at regular intervals.
2. The system shall prevent accidental deletion of uploaded scripts without user confirmation.
3. The system shall preserve original answer scripts even if AI processing fails.
4. The system shall maintain system logs for uploads, evaluations, and manual modifications.
5. The system shall implement proper error-handling mechanisms to prevent data corruption.
6. The system shall support data recovery in the event of system failure.
7. The system should notify administrators in case of critical failures.

5.3 Security Requirements

The MarkNex system must protect user credentials and student academic records:

1. The system shall enforce secure authentication using username and password.
2. The system shall store passwords using secure hashing and salting techniques.
3. The system shall implement role-based access control for Teachers and Administrators.
4. The system shall ensure that users can only access data related to their assigned classes.

5. All data transmission between client and server shall use HTTPS encryption.
6. The system shall automatically terminate inactive sessions after a defined timeout period.
7. The system shall log all login attempts, uploads, and manual mark changes.
8. The system must prevent unauthorized access to student records.
9. The system should implement protection against common web vulnerabilities (e.g., SQL injection, cross-site scripting).

5.4 Software Quality Attributes

The MarkNex system shall satisfy the following quality attributes:

- Usability
 1. The user interface shall be intuitive and easy to navigate.
 2. Teachers should be able to upload scripts and view results without technical training.
 3. The dashboard shall present analytics in a clear and readable format.
- Reliability
 1. The system shall operate without critical failure during normal usage.
 2. The system shall ensure consistent AI evaluation accuracy.
 3. The system shall handle unexpected errors gracefully without crashing.
- Maintainability
 1. The system shall follow modular architecture principles.
 2. The source code shall be documented for future maintenance.
 3. The AI module should be upgradeable without affecting other components.
- Portability
 1. The system shall support modern web browsers (Chrome, Firefox, Edge).
 2. The system should be deployable on both cloud and on-premise environments.
- Interoperability
 1. The system should allow integration with external Learning Management Systems (LMS).
 2. The system shall generate reports in standard formats such as PDF.

5.5 Business Rules

The following business rules shall govern the operation of the MarkNex system:

1. Only registered Teachers and Administrators shall be allowed to access the system.
2. Each uploaded answer script must be linked to a valid student identifier.
3. The AI module shall assign marks according to predefined marking schemes.

4. Teachers shall have the authority to modify AI-assigned marks when necessary.
5. The system shall automatically recalculate total marks after any modification.
6. Answers with confidence scores below the defined threshold must be flagged for manual review.
7. Historical evaluation records shall not be permanently deleted without administrative approval.
8. The confidence threshold value shall be configurable by the Administrator.
9. Reports generated by the system shall comply with institutional grading policies.

6. Other Requirements

This section defines additional requirements that are not covered elsewhere in the SRS, including database, legal, scalability, and future expansion requirements.

- **Database Requirements** – The system shall use a secure relational or cloud-based database to store user credentials, uploaded scripts, AI evaluation results, confidence scores, and audit logs. The database must ensure data consistency, backup, and recovery mechanisms.
- **Internationalization** – Future versions of the system should support multilingual interfaces to allow users to select their preferred language.
- **Legal & Compliance Requirements** – The system must comply with institutional academic regulations and applicable data protection and privacy laws. Student records must be handled confidentially and must not be shared without authorization.
- **Scalability Requirements** – The system should support increasing numbers of users and answer scripts without significant degradation in performance. The AI processing module should be scalable to handle larger workloads.
- **Audit & Logging Requirements** – The system must maintain logs of login attempts, file uploads, AI evaluations, and manual mark modifications for accountability and traceability.
- **Reuse Objectives** – The modular design of the system should allow components such as authentication, file management, and AI evaluation modules to be reused or extended in future academic assessment systems.

Appendix A: Glossary

This section defines key terms, acronyms, and abbreviations used in the MarkNex SRS to ensure a common understanding among all stakeholders.

1. **AI:** Artificial Intelligence.
2. **API:** Application Programming Interface, used for communication between the backend and the Multimodal LLM.

3. **Confidence Score:** A numerical value generated by the AI indicating the statistical certainty of the assigned mark.
4. **DFD:** Data Flow Diagram, used to visualize the flow of information within the system.
5. **Human-in-the-Loop:** A model of interaction where the AI performs tasks but requires human intervention (the teacher) for validation or error correction.
6. **LLM:** Large Language Model; specifically, a Multimodal LLM that can process both text and images (scanned scripts).
7. **Marking Scheme:** A predefined set of criteria or model answers used to evaluate student responses.
8. **Multimodal:** The ability of an AI model to process different types of data (e.g., images of handwriting and digital text) simultaneously.
9. **OCR:** Optical Character Recognition (though MarkNex aims to exceed traditional OCR limitations).
10. **RDBMS:** Relational Database Management System (e.g., MySQL or PostgreSQL).
11. **SRS:** Software Requirements Specification.
12. **UI/UX:** User Interface and User Experience.

Appendix B: Analysis Models

This appendix lists the pertinent analysis models used to visualize the structure, data movement, and logic of the MarkNex system.

1. **Data Flow Diagrams (DFD):** Level 0 (Context Diagram) and Level 1 Diagrams showing the movement of script data from upload to final report generation.
2. **Use Case Diagram:** Illustrates the interactions between the Teacher, the AI Evaluation Engine, and the system boundaries.
3. **Entity-Relationship Diagram (ERD):** Visualizes the database schema, including relationships between Students, Answer Scripts, Marks, and Teachers.
4. **System Architecture Diagram:** High-level overview of the React frontend, Node.js/Python backend, and the integration with the external LLM API.
5. **High-Level System Workflow:** A sequential diagram representing the process from authentication to manual review and final storage.

Appendix C: To Be Determined List

This section tracks unresolved decisions or missing technical details that will be finalized during the development phase.

1. **Specific Multimodal LLM Selection:** Final choice between specific models (e.g., GPT-4o, Claude 3.5 Sonnet, or Gemini 1.5 Pro) based on performance with Sinhala/English handwriting.
2. **Confidence Threshold Value:** The exact percentage (e.g., 70% or 80%) that triggers a "Low-Confidence" flag for manual review.

3. **LMS Integration API:** Specific protocols for future integration with existing University or School Learning Management Systems.
4. **Cloud Service Provider:** Final selection of hosting (AWS, Google Cloud, or Azure) based on GPU availability for AI inference.
5. **Storage Limits:** Specific storage quota per teacher for high-resolution scanned PDF/JPG files.
6. **Handwriting Versatility:** Limitations and supported styles for the initial version of the Grade 6 Mathematics scripts.